
Quick Recap of Session 1

- A **Proxy** wraps a real object to add behavior without touching its code — you built one by hand with `InvocationHandler` in Lab 1
- **Spring AOP** auto-generates proxies (JDK dynamic proxy or CGLIB)
- **AspectJ** = the full, standalone AOP framework (compile-time/load-time weaving, any join point)
- **Spring AOP** = a simpler, proxy-based subset — only advises Spring bean method calls
- We wrote **Advice** as plain interfaces: `MethodBeforeAdvice`, `AfterReturningAdvice`, `ThrowsAdvice`, `MethodInterceptor`
- **Advice + Pointcut = Advisor**; `ProxyFactory` / `ProxyFactoryBean` built the proxy

Today: same underlying mechanism, much nicer syntax — **annotations**.

Session 2 Agenda

1. From interfaces to annotations: meet `@Aspect` and the 5 advice annotations
 2. Pointcut expressions, properly — including step-by-step `execution()` matching
 3. Reusable pointcuts & combining them
 4. Ordering multiple aspects
 5. Real-world aspects: custom security, caching
 6. How Spring itself uses AOP (`@Transactional`)
 7. Limitations (self-invocation!)
 8. **Lab 2** — practice every annotation from this session
-

Setting Up Annotation-Style Spring AOP (Still Spring Core)

Maven — one more artifact than last time, still no Spring Boot:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.0</version>
```

```

</dependency>
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>6.1.0</version>
</dependency>
<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.9.21</version>
</dependency>

```

aspectjweaver isn't there to *weave* anything — remember, we're still on Spring AOP proxies. We only need it because it's the jar that **defines** the `@Aspect`, `@Before`, `@Around` annotation types and the `execution(...)` pointcut expression parser that Spring AOP reuses.

Since we have no Spring Boot auto-configuration, we must turn annotation scanning **on** explicitly:

```

@Configuration
@EnableAspectJAutoProxy    // <-- tells Spring: "scan for @Aspect beans and
                           proxy them"
@ComponentScan("com.example")
public class AppConfig { }

```

From Interfaces to Annotations — Same Idea, New Syntax

Last session, "before advice" meant a whole class:

```

public class LoggingBeforeAdvice implements MethodBeforeAdvice {
    public void before(Method method, Object[] args, Object target) {
        System.out.println("BEFORE: " + method.getName());
    }
}

```

Today, the same behavior is one annotated method inside an `@Aspect` class:

```

@Aspect
@Component
public class LoggingAspect {

```

```

@Before("execution(* com.example.service.AccountService.*(..))")
public void logBefore(JoinPoint joinPoint) {
    System.out.println("Calling: " + joinPoint.getSignature().getName());
    System.out.println("Args: " + Arrays.toString(joinPoint.getArgs()));
}
}

```

- `@Aspect` → marks the class as an aspect (conceptually the same role as your `LoggingBeforeAdvice` class)
- `@Component` → so Spring picks it up and manages it as a bean
- `@Before` → the **advice type** — instead of implementing an interface, you just name it
- the string → a **pointcut expression** (`execution(...)`) — where interfaces made you write a `Pointcut` object, now you write a one-line expression
- `JoinPoint` → a friendlier version of the raw `Method / args / target` you handled manually before

No `ProxyFactory` call anywhere — `@EnableAspectJAutoProxy` finds every `@Aspect` bean and builds the proxies for you automatically.

The 5 Annotation Advice Types — Quick Map to What You Know

Annotation	Classic interface equivalent	Runs
<code>@Before</code>	<code>MethodBeforeAdvice</code>	before the method
<code>@AfterReturning</code>	<code>AfterReturningAdvice</code>	after success, has the return value
<code>@AfterThrowing</code>	<code>ThrowsAdvice</code>	after an exception, has the exception
<code>@Around</code>	<code>MethodInterceptor</code>	wraps the whole call — you call <code>proceed()</code>
<code>@After</code>	<i>(no classic equivalent — needed try/finally inside <code>MethodInterceptor</code> before)</i>	always, success or failure

`@After` is the one genuinely **new** advice type today — annotations gave it its own name instead of making you write `try/finally` by hand.

Advice Type 1: @Before

Runs **before** the target method executes.

```
@Before("execution(* com.example.service.*.*(..))")
public void logBefore(JoinPoint jp) {
    System.out.println("BEFORE: " + jp.getSignature());
}
```

Typical uses: logging, authorization checks, input validation.

Advice Type 2: @After (finally)

Runs **after** the method finishes — **whether it succeeded or threw an exception** (like a `finally` block). This is the one advice type with no classic-interface equivalent.

```
@After("execution(* com.example.service.*.*(..))")
public void logAfter(JoinPoint jp) {
    System.out.println("AFTER (always runs): " + jp.getSignature());
}
```

Advice Type 3: @AfterReturning

Runs only if the method **completed successfully**. Can access the return value.

```
@AfterReturning(
    pointcut = "execution(* com.example.service.*.*(..))",
    returning = "result"
)
public void logSuccess(JoinPoint jp, Object result) {
    System.out.println("SUCCESS: " + jp.getSignature() + " returned " +
        result);
}
```

Advice Type 4: @AfterThrowing

Runs only if the method **threw an exception**.

```

@AfterThrowing(
    pointcut = "execution(* com.example.service.*(..))",
    throwing = "ex"
)
public void logFailure(JoinPoint jp, Exception ex) {
    System.out.println("ERROR in " + jp.getSignature() + ": " +
ex.getMessage());
}

```

Great for centralized error logging/alerting.

Advice Type 5: @Around — The Powerful One

Wraps the **entire** method call — the annotation-style twin of `MethodInterceptor`.

```

@Around("execution(* com.example.service.*(..))")
public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
    long start = System.currentTimeMillis();
    System.out.println("START: " + pjp.getSignature());

    Object result = pjp.proceed(); // <-- actually calls the real method!

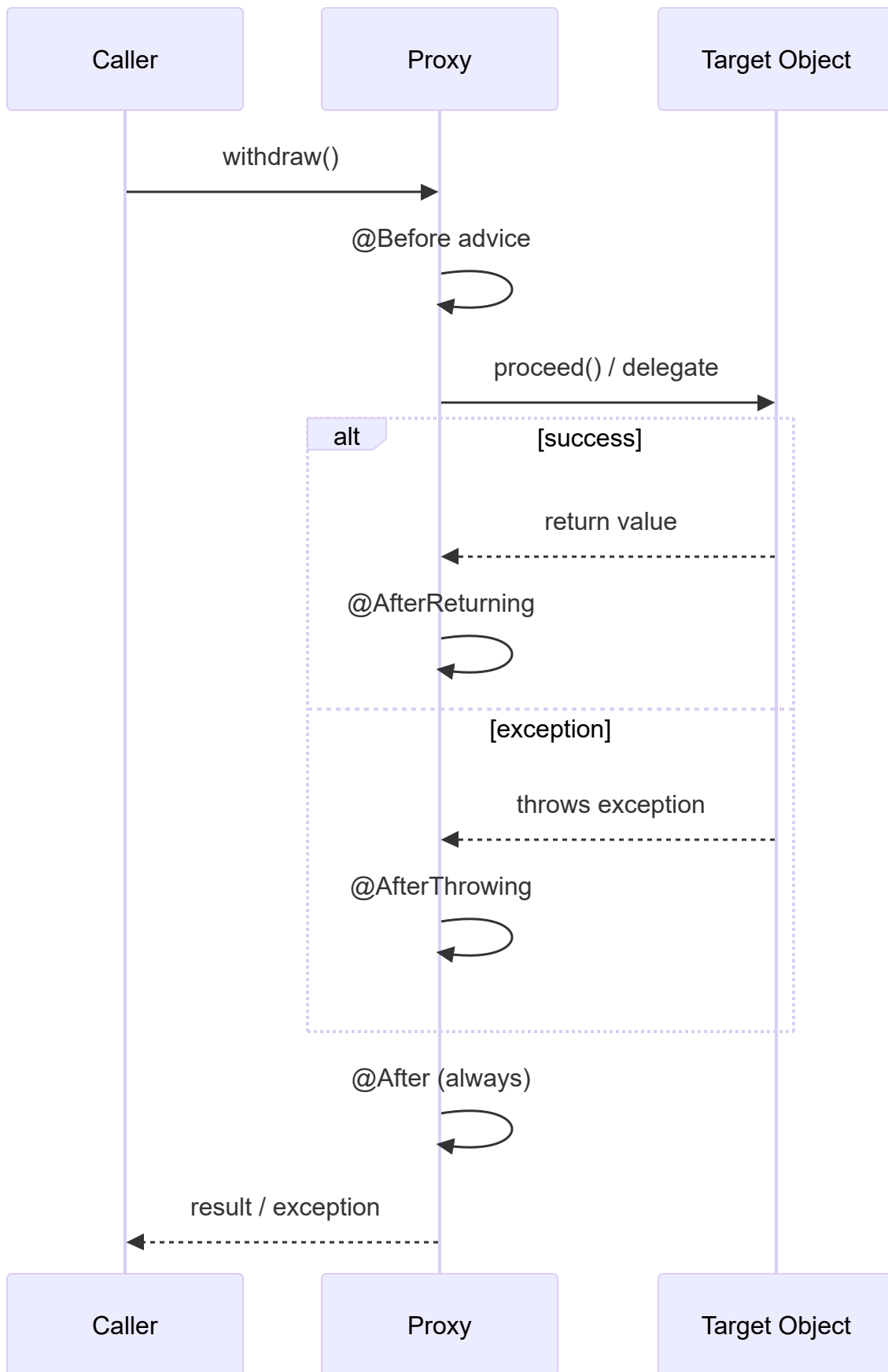
    System.out.println("END: " + pjp.getSignature() +
        " took " + (System.currentTimeMillis() - start) + "ms");
    return result;
}

```

`ProceedingJoinPoint` is just `JoinPoint` plus a `proceed()` method — the annotation-style equivalent of `MethodInvocation.proceed()`.

⚠ Forgetting to call `pjp.proceed()` means the **real method never runs** — exactly the same trap as forgetting `invocation.proceed()` last session.

All 5 Advice Types — Timeline View



`@Around` wraps this *entire* diagram — it's the only one that can run code both before **and** after, and even skip the call entirely.

Full Example: A Complete Logging Aspect

```
@Aspect
@Component
public class ServiceLoggingAspect {

    private static final Logger log =
LoggerFactory.getLogger(ServiceLoggingAspect.class);

    @Around("execution(* com.example.service.*(..))")
    public Object logExecution(ProceedingJoinPoint pjp) throws Throwable {
        String method = pjp.getSignature().toShortString();
        log.info("-> Entering {} with args {}", method,
Arrays.toString(pjp.getArgs()));
        long start = System.currentTimeMillis();
        try {
            Object result = pjp.proceed();
            log.info("<- Exiting {} in {}ms, result={}",
                method, System.currentTimeMillis() - start, result);
            return result;
        } catch (Throwable ex) {
            log.error("!! Exception in {}: {}", method, ex.getMessage());
            throw ex;
        }
    }
}
```

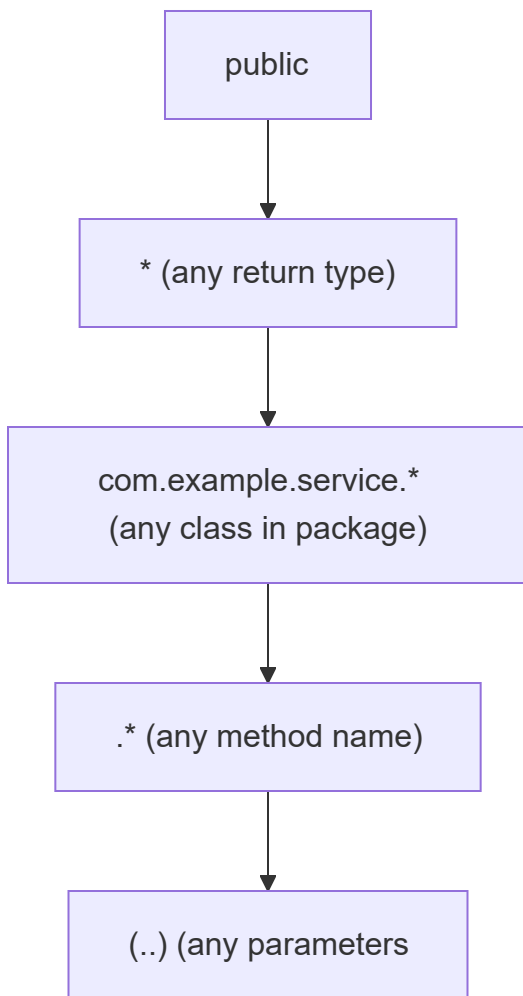
Notice: `AccountService` **still has zero logging code in it** — same promise as the classic interfaces, friendlier syntax.

Anatomy of `execution()`

The most common pointcut expression. Full syntax:

```
execution(modifiers-pattern? ret-type-pattern declaring-type-pattern?
          method-name-pattern(param-pattern) throws-pattern?)
```

```
execution(public * com.example.service.*.*(..))
```



execution() — Worked Examples

Expression	Matches
<code>execution(* com.example.service.AccountService.withdraw(..))</code>	exactly <code>withdraw</code> , any args, in that one class
<code>execution(* com.example.service.AccountService.*(..))</code>	any method of <code>AccountService</code>
<code>execution(* com.example.service...*(..))</code>	any method, any class, in <code>service</code> and all sub-packages (note the <code>..</code>)
<code>execution(public String com.example...*.find*(..))</code>	public methods returning <code>String</code> , name starting with <code>find</code>
<code>execution(* *(String, ..))</code>	any method whose first param is a <code>String</code>

How Spring Actually Matches an `execution()` Pointcut

Given a real call — `accountServiceImpl.withdraw("acc-1", 100.0)` — and a pointcut, Spring checks the signature **piece by piece, left to right**. If any piece fails, the whole thing fails; it doesn't try to be clever.

```
execution(public double com.example.service.AccountService.withdraw(String, double))
```

1. **modifiers** — is the method `public` ?
2. **return type** — does it return `double` ?
3. **declaring type** — is it declared on `com.example.service.AccountService` ?
4. **method name** — is it called `withdraw` ?
5. **parameters** — exactly `(String, double)` ?
6. **throws** (*omitted here → not checked*)

All six checks pass → **this join point matches**, and the advice fires. Change any single piece — return type, name, one parameter type — and it stops matching.

The Full `execution()` Signature, Piece by Piece

```
execution(modifiers-pattern? ret-type-pattern declaring-type-pattern?  
          method-name-pattern(param-pattern) throws-pattern?)
```

Segment	Required?	Example	Meaning
modifiers-pattern	optional	<code>public</code>	restrict by visibility
ret-type-pattern	required	<code>double</code> , <code>*</code>	exact type, or <code>*</code> = any
declaring-type-pattern	optional	<code>AccountService</code>	which class/interface; omit = any
method-name-pattern	required	<code>withdraw</code> , <code>with*</code> , <code>*</code>	exact, wildcard, or any
param-pattern	required	<code>(String, double)</code> , <code>(..)</code>	see next slide
throws-pattern	optional	<code>throws Exception</code>	rarely used

Only **return type**, **method name**, and **parameters** are mandatory — everything else can be left out, and Spring treats the missing piece as "match anything."

The Two Wildcards — * vs ..

These mean **different things** depending on where you put them, and mixing them up is the most common `execution()` mistake.

Wildcard	Matches	Used in
*	exactly one segment — one package level, one whole type, part of a name	return type, one package level, part of a method name
..	zero or more of something	remaining package depth, or any number of parameters

within() — Match by Type, Not Method Signature

Simpler than `execution()` when you just want "everything in this class/package":

```
@Before("within(com.example.service.*)") // any class directly in this
package
public void logService(JoinPoint jp) { ... }

@Before("within(com.example.service..*)") // package AND all sub-packages
public void logServiceDeep(JoinPoint jp) { ... }
```

@annotation() — Pointcuts Driven by Custom Annotations

This is how you build **your own declarative features** — like Spring's own `@Transactional`.

Step 1 — define the annotation:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface LogExecutionTime { }
```

Step 2 — write the aspect:

```
@Around("@annotation(com.example.aop.LogExecutionTime)")
public Object timeIt(ProceedingJoinPoint pjp) throws Throwable {
    long start = System.nanoTime();
    Object result = pjp.proceed();
    System.out.println((System.nanoTime() - start) / 1_000_000 + "ms");
    return result;
}
```

Step 3 — use it anywhere:

```
@LogExecutionTime
public void heavyComputation() { ... }
```

Combining Pointcuts

Use `&&`, `||`, `!` just like boolean logic:

```
@Before("execution(* com.example.service.*(..)) " +
        "&& @annotation(com.example.aop.Audited) " +
        "&& !execution(* *.getHealth(..))")
public void auditExceptWhenHealthCheck(JoinPoint jp) { ... }
```

Reads as: *"apply to service methods, that are `@Audited`, except `getHealth`"*.

Reusable Pointcuts with `@Pointcut`

Don't repeat the same long expression in every advice — name it once:

```
@Aspect
@Component
public class CommonPointcuts {

    @Pointcut("execution(* com.example.service.*(..))")
    public void serviceLayer() {}

    @Pointcut("@annotation(com.example.aop.Audited)")
```

```
    public void audited() {}  
}
```

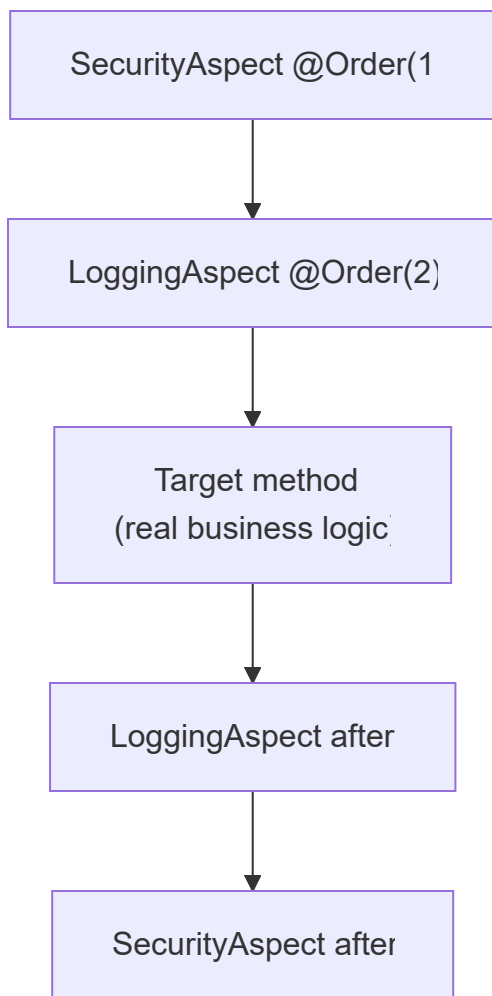
```
@Aspect  
@Component  
public class AuditAspect {  
    @Before("com.example.aop.CommonPointcuts.serviceLayer() &&  
com.example.aop.CommonPointcuts.audited()")  
    public void audit(JoinPoint jp) { ... }  
}
```

Multiple Aspects — Who Runs First?

If several aspects apply to the same method, order matters (e.g., you want **security** to run before **logging**).

```
@Aspect  
@Order(1)    // lower number = higher priority = runs first (outermost)  
@Component  
public class SecurityAspect { ... }  
  
@Aspect  
@Order(2)  
@Component  
public class LoggingAspect { ... }
```

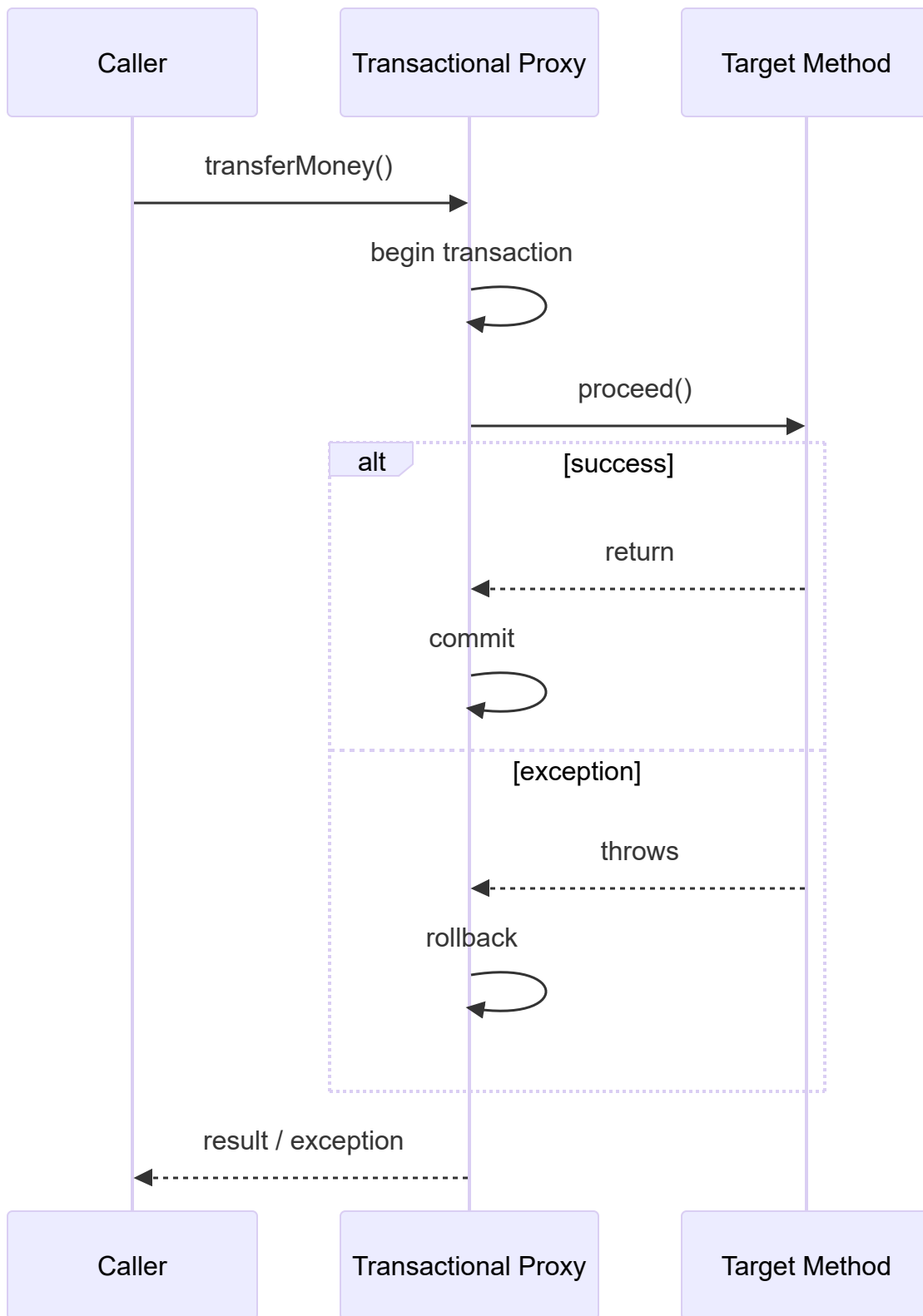
The "Onion" Model of Multiple Aspects



Each `@Around` aspect wraps the next one, like layers of an onion — the lowest `@Order` value is the **outermost** layer.

Real-World Aspect #2: How Spring's Own `@Transactional` Works

`@Transactional` is itself implemented as an **AOP** `@Around` **advice** internally:



This is exactly the `@Around` pattern you already know — Spring just ships the aspect for you. Recognizing this pattern is a big "aha" moment.

Real-World Aspect #3: Simple Custom Caching

```

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Cacheable { }

@Aspect
@Component
public class CachingAspect {

    private final Map<String, Object> cache = new ConcurrentHashMap<>();

    @Around("@annotation(Cacheable)")
    public Object cache(ProceedingJoinPoint pjp) throws Throwable {
        String key = pjp.getSignature().toShortString() +
Arrays.toString(pjp.getArgs());
        if (cache.containsKey(key)) {
            System.out.println("CACHE HIT: " + key);
            return cache.get(key);
        }
        Object result = pjp.proceed();
        cache.put(key, result);
        return result;
    }
}

```

(Spring actually ships a real `@Cacheable` that works this way — with a proper cache store.)

Limitations — Self-Invocation

This is the **#1 surprise** for people new to Spring AOP:

```

@Service
public class OrderService {

    @Cacheable
    public Order getOrder(String id) { ... }

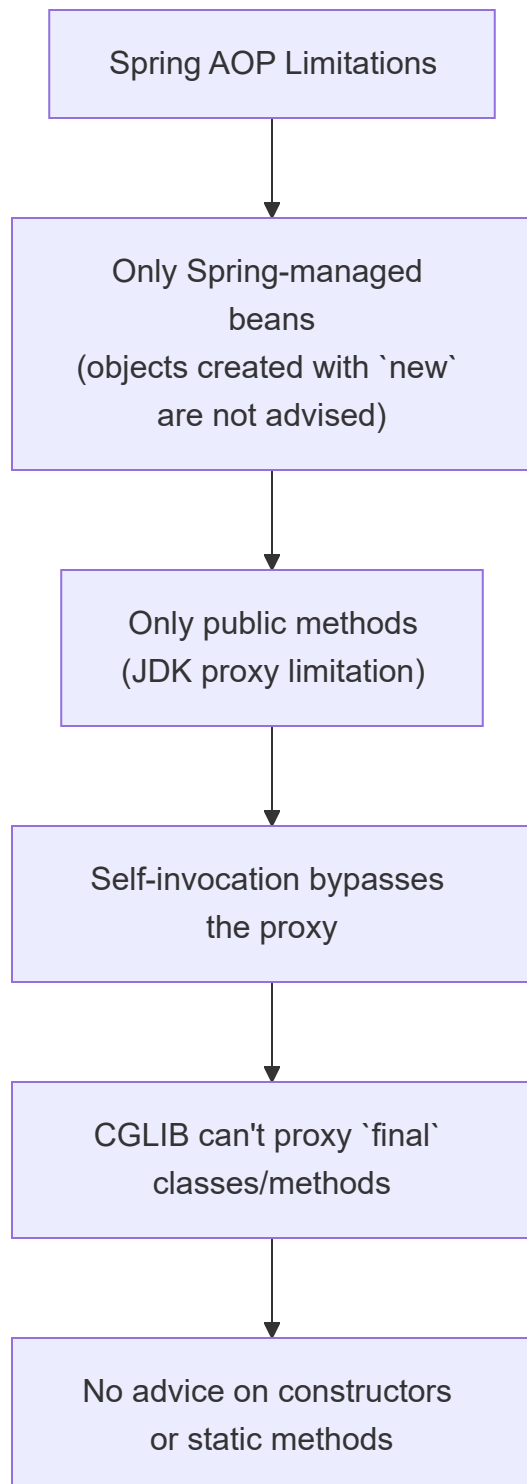
    public void processOrder(String id) {
        Order o = getOrder(id); // ⚠ NOT going through the proxy!
        // caching advice will NOT run here
    }
}

```

Why: `this.getOrder(id)` is a plain Java call on `this` — it never passes back through the Spring-generated proxy, so no advice fires.

Fixes: inject the proxy of itself (`@Autowired self` pattern), split into two beans, or use full AspectJ compile-time weaving.

Limitations — The Full List



If you truly need these, that's when you reach for **full AspectJ** (compile-time or load-time weaving) instead of Spring AOP.

Debugging Tip

Not sure if a bean is actually proxied? Check its class at runtime:

```
System.out.println(accountService.getClass());  
// interface-based -> something like com.sun.proxy.$Proxy23  
// class-based      -> something like AccountServiceImpl$$SpringCGLIB$$0
```

If it just prints your plain class name with no \$\$ or \$Proxy, **no proxy was created** — check your pointcut expression matches, and that the bean is actually managed by Spring.

Session 2 Recap

- `@Aspect` + 5 advice annotations (`@Before` , `@After` , `@AfterReturning` , `@AfterThrowing` , `@Around`) — the annotation-style twins of the interfaces from Session 1
 - `execution()` , `within()` , `@annotation()` , `args()` , `bean()` — different ways to say "where"
 - `@Pointcut` for reuse, `&&` / `||` / `!` to combine
 - `@Order` controls layering when multiple aspects apply
 - Spring's own `@Transactional` / `@Cacheable` are built on exactly this mechanism
 - Self-invocation is the #1 gotcha — the call must go **through the proxy**
-

Lab 2 — Practice Every Annotation

Part A — the 5 core advice annotations (no custom annotations yet):

1. `@Before`
2. `@AfterReturning`
3. `@AfterThrowing`
4. `@After`
5. `@Around` : replace all four above with a single around-advice.

Part B — build your own declarative features with `@annotation()` :

6. Create a custom `@Cacheable` Example